Guía autoguiada · Practicar NoSQL en parejas (cuaderno S03_01)

PARA QUÉ SIRVE

os explica **cómo trabajar vosotros dos, paso a paso**, el cuaderno de NoSQL del Sprint 3 (`S03_01_nosql_mongomock`), sin que nadie tenga que deciros qué hacer en cada momento.

- **Tiempo en clase:** unos **30 minutos** (hasta el ejercicio 5). Lo demás (ejercicios 6 y 7, mini-reto y preguntas) es **práctica para casa**.
- **Qué necesitáis:** un navegador y una cuenta de Google (para ejecutar el cuaderno en **Colab**), o Python con Jupyter si preferís trabajar en local. **No hay que instalar MongoDB**: usamos `mongomock`, una imitación en memoria.
- **Qué no necesitáis:** saber programar a fondo. Casi todo es **rellenar una línea** donde pone `....`
- **Datos:** son **sintéticos** (inventados). Son las mismas reservas de TurisData que en SQL, pero guardadas como **documentos**.

1. Antes de empezar (5 minutos)

1. **Formad parejas** y decidid quién **conduce** primero (el otro **navega**; sección 2). Se trabaja en **una sola pantalla**.
2. **Abrid el cuaderno.** Pulsad la insignia **Abrir en Colab** del README del Sprint 3 o abrid este enlace directo.
   - **Guardad una copia suya:** *Archivo → Guardar una copia en Drive*. Así no perdéis lo que escribáis.
   - **En local:** descargad el cuaderno y abridlo con Jupyter o VS Code.
3. **No hace falta descargar la base de datos.** La primera celda de código descarga `turisdata.db` ella sola (necesita internet).
4. **Ejecutad las celdas de arriba abajo, de una en una** (Mayús+Intro ejecuta la celda y pasa a la siguiente). **No uséis «Ejecutar todo»:** los ejercicios llevan `...` y darían error hasta que los rellenéis.
5. **Haced la prueba de arranque.** Tras ejecutar la celda de **«2 · Idea: insertar en una colección»** debe salir:

```
Documentos en la colección: 6000
```

Si sale otro número o un error, parad y preguntad antes de seguir. Si Colab os dice que falta `mongomock`, **ejecutad otra vez** la celda de librerías (la que empieza por `# Librerías`).

Cifras de control (para saber que estáis en la colección correcta):

Qué contáis	Debe salir
Documentos (reservas)	**6.000**
Primera reserva construida (`_id`)	**5760**, de Tenerife, 3 noches, 89.24 €
Canales distintos	**4**
Islas distintas	**6**

> El texto del cuaderno enseña un documento de ejemplo con `"pais": "UK"`. Es **solo una ilustración**: el documento que imprime vuestra celda es el real (su país será `Nordicos`).

2. Cómo trabajáis en pareja: conductor y navegante

Rol	Qué hace	Qué no hace
Conductor/a	Escribe en el cuaderno y ejecuta las celdas	No decide solo: **explica en voz alta** lo que escribe
Navegante	Lee el enunciado, **revisa** lo que se escribe y vigila los `$` y las comillas	No dicta letra por letra ni toca el teclado

- **Cambiad de rol** en cada ejercicio (o cada 5 minutos).
- **Pedid ayuda así:** primero preguntáos entre vosotros, luego a la pareja de al lado, y **después** a vuestro docente. Llevad **la línea que habéis probado y el mensaje de error**.

3. El ciclo de cada ejercicio (6 pasos)

Repetid esto **en cada ejercicio**. Con la práctica os llevará menos de 5 minutos.

1. **Leed** el enunciado **en voz alta**, los dos. Fijaos en **qué variable** hay que rellenar (por ejemplo `n_canceladas`).

2. **Predecid** antes de escribir: *¿qué me piden: un número, una lista o un diccionario? ¿Cuántos resultados espero (pocos, muchos, uno)?*
3. **Mirad el ejemplo resuelto** de arriba (cada apartado tiene uno) y **cambiadlo** poco a poco para el ejercicio. Sustituid los `...` y dejad la línea del `TODO` como pista.
4. **Ejecutad la celda.** Si todo va bien, sale **«Correcto»**.
5. **Si falla:** leed el mensaje. Es un `assert` (una comprobación) y **explica qué mirar**; no es un error grave. Revisad con la lista de abajo.
6. **Explicad a vuestra pareja** con una frase qué hace cada parte del filtro. Si no sabéis explicar una parte, **todavía no la habéis entendido**.

Lista de comprobación (paso 5)

- ☐ **¿Habéis rellenado la variable con el nombre exacto** que pide el enunciado?
- ☐ **¿Cada operador lleva su `$`?** (`$gt`, `$or`, `$inc`, `$group`…).
- ☐ **¿Los textos van entre comillas y escritos igual** que en los datos (`"Tenerife"`, `"web_intermediaria"`)?
- ☐ **¿Habéis escrito `True`, con mayúscula inicial,** y no `true` en los campos de sí/no?
- ☐ **Para subdocumentos:** ¿habéis usado el punto (`"alojamiento.isla"`) entre comillas?
- ☐ **Si pedían una lista:** ¿habéis envuelto el cursor con `list(...)`?

Cómo saber si el resultado es correcto, **antes de mirar las soluciones**

- **Leed el mensaje del `assert`.** Dice qué número se esperaba y cuántos habéis obtenido. Si sale 0, casi siempre es un filtro mal escrito.
- **Probad una versión más fácil.** Antes de pedir «de Tenerife **y** de más de 150 euros», probad solo «de Tenerife».
- **Contad.** `count_documents(mismo_filtro)` os dice cuántos documentos hay antes de borrar o cambiar nada.
- **Mirad un documento a ojo** con `find_one(filtro)` para ver si es lo que pedía el enunciado.
- **Comprobad con otro camino.** En el mini-reto, SQL y MongoDB **tienen que dar lo mismo**.

---

4. Plan de la sesión (30 minutos en clase + casa)

Parte del cuaderno	Minutos	Qué hacéis
1 y 2 · Documentos y colección	8	Leer la idea, ejecutar las celdas y llegar a **6.000 documentos**
Ejercicio 1 · contar	3	`count_documents`
3 · `find` + ejercicios 2 y 3	9	Filtros con `$gt` y `$or`
4 · Actualizar + ejercicio 4	5	`update_one` con `$inc`
Ejercicio 5 · borrar con cuidado	5	Contar antes de borrar
Aquí termina la clase		
5 · Agregaciones + ejercicios 6 y 7	en casa	`$group`, `$match`, `$sort`
6 · Arrays y **7 · mini-reto**	en casa	`$unwind` y la misma pregunta en SQL y MongoDB
8 · Para reflexionar	en casa	Tres preguntas para el cuaderno de aprendizaje

Si vais con prisa: haced **solo los ejercicios 1 a 4**. **Si vais sobrados:** pasad a las agregaciones y **explicad a otra pareja** cómo funciona un `$group`.

---

5. Cada parte en una página

> Aquí tenéis **la idea de cada parte**, un **esqueleto** de otro caso y **qué se suele hacer mal**. **No** están las soluciones: las construís vosotros. Cuando hayáis terminado (o llevéis mucho rato atascados), podéis contrastar con el **solucionario paso a paso**. El listado completo está en `listado_ejercicios_nosql.md`.

Ejercicios 1 y 5 · Contar y borrar con cuidado

Idea clave: `count_documents(filtro)` **cuenta**; `delete_many(filtro)` **borra**. Siempre se **cuenta antes con el mismo filtro**.

Esqueleto:

```
filtro = {"canal": "agencia", "noches": {"$gte": 12}}   # el mismo filtro para contar y para borrar
cuantos = reservas.count_documents(filtro)
print(cuantos)   # mirad el número ANTES de borrar
```

Se suele hacer mal:

- **Escribir `True` como `true`:** Python da `NameError`. Escribid `True`, con la primera letra en mayúscula.
- **Borrar sin filtro** (`delete_many({})`): se llevaría **los 6.000 documentos**.
- **Ejecutar el borrado dos veces:** la segunda vez ya no queda nada que borrar y el `assert` falla. Si pasa, volved a ejecutar la celda que carga los 6.000 documentos y repetid el ejercicio.

Comentad en pareja: *¿qué diferencia hay entre `delete_one` y `delete_many`?*

Ejercicios 2 y 3 · Filtrar con `find` y operadores

Idea clave: un filtro es un **diccionario**. Varias condiciones separadas por coma son un **Y**. Para un **O** se usa `$or` con una lista.

Esqueleto:

```
largas = list(reservas.find({"alojamiento.categoria": "hostal", "noches": {"$gte": 7}}))
print(len(largas))
```

Se suele hacer mal:

- **Olvidar el `$`** en los operadores (`{"noches": {"gt": 5}}` no funciona).
- **Olvidar las llaves de dentro:** es `{"noches": {"$gt": 5}}`, no `{"noches": "$gt": 5}`.
- **Olvidar `list(...)`:** `find` devuelve un **cursor**, no una lista.
- **Escribir el campo del subdocumento sin el punto o sin comillas.**

Comentad en pareja: *¿cómo escribiríais en SQL el filtro del ejercicio 3?*

Ejercicio 4 · Actualizar un documento

Idea clave: `update_one(filtro, {"$set": {...}})` **cambia un campo**; `$inc` **suma** a un número.

Esqueleto:

```
reservas.update_one({"_id": 1}, {"$set": {"canal": "directo"}})
print(reservas.find_one({"_id": 1}, {"canal": 1}))   # comprobad siempre después
```

Se suele hacer mal:

- **Ejecutar el `$inc` dos veces:** el precio sube **10 en vez de 5**. Si pasa, volved a ejecutar la celda de la sección 2 (la que carga los 6.000 documentos) y repetid el ejercicio.
- **Olvidar el `$set` o el `$inc`:** sin ellos da un error, porque `update_one` solo trabaja con operadores que empiezan por `$`.

Comentad en pareja: *¿qué diferencia hay entre `update_one` y `update_many`?*

Ejercicios 6 y 7 · Agregaciones (en casa)

Idea clave: una **pipeline** es una **lista de etapas**: `$match` (filtrar) → `$group` (agrupar) → `$sort` (ordenar). Equivale a `WHERE` → `GROUP BY` → `ORDER BY`.

Esqueleto:

```
pipeline = [
    {"$match": {"tarifa": "flexible"}},
    {"$group": {"_id": "$deposito", "total": {"$sum": 1}}},
    {"$sort": {"total": -1}},
]
print(list(reservas.aggregate(pipeline)))
```

Se suele hacer mal:

- **Olvidar el `$` delante del nombre del campo** dentro del grupo (`"$deposito"`, no `"deposito"`).
- **Poner las etapas en otro orden:** primero se filtra y luego se agrupa.
- **Escribir `_id` mal:** en `$group` el `_id` es **la clave de agrupación**, no el identificador de la reserva.

Comentad en pareja: *¿qué dos etapas de la pipeline equivalen a `WHERE` y a `GROUP BY`?*

Parte 6 y mini-reto · Arrays y la misma pregunta en SQL y MongoDB (en casa)

Idea clave: un documento puede llevar una **lista** (las experiencias de una reserva). `$unwind` la «desenrolla» para contarla. En el mini-reto, **la respuesta debe ser la misma** con SQL y con MongoDB.

Se suele hacer mal:

- **Redondear distinto en SQL y en MongoDB:** si los dos dan `46.0` y `46.04`, no coinciden en el `assert`. Redondead **los dos** a `1` decimal.
- **Comparar diccionarios con el orden distinto:** da igual el orden, importa que **las islas y los valores** sean los mismos.

---

6. Si algo falla

Qué veis	Qué suele pasar	Qué hacer
`NameError: name 'n_canceladas' is not defined`	No habéis ejecutado la celda del ejercicio (o dejasteis `...`)	Rellenad la variable y ejecutad **la celda del ejercicio**
`AssertionError: Revisa el filtro...`	El filtro devuelve otro número	Leed el mensaje: **dice cuántos esperaba**. Revisad `$`, comillas y `True`
0 resultados	Texto con otra mayúscula o campo mal escrito	Probad `reservas.find_one()` para ver los nombres de los campos
`TypeError: ... Cursor`	Os falta `list(...)`	Envolved el `find(...)` con `list(...)`
`ModuleNotFoundError: mongomock`	Falta instalarla	Ejecutad **otra vez** la celda de librerías (necesita internet)
Falla la descarga de `turisdata.db`	Sin internet o sesión nueva	Comprobad la conexión y ejecutad de nuevo la celda de **Preparación**
Lo que cambiasteis **ha desaparecido**	Colab reinició la sesión	Ejecutad **desde la celda de Preparación** hasta donde estabais
Se ha borrado o cambiado **de más**	Faltaba el filtro	Ejecutad de nuevo la celda que carga los 6.000 documentos y repetid

---

7. Al terminar

Lista de «ya sé...» (marcad entre los dos; si alguna casilla queda en blanco, repasadla)

- ☐ **Sé explicar qué es un documento** y en qué se diferencia de una fila de SQL.
- ☐ **Sé contar y filtrar** con `count_documents` y `find`, usando `$gt`, `$or` y el punto para subdocumentos.
- ☐ **Sé cambiar y borrar** con `$set`, `$inc` y `delete_many`, comprobando antes con el mismo filtro.
- ☐ **Sé escribir una pipeline** con `$match`, `$group` y `$sort`.
- ☐ **Sé decir cuándo usaría MongoDB** y cuándo seguiría con SQL.

Tres preguntas para hablar entre los dos (2 minutos)

1. **¿Qué ejercicio nos costó más** y qué nos ayudó a resolverlo?
2. **¿Qué ventaja tiene** guardar el alojamiento dentro de cada reserva? ¿Y qué problema si cambia de nombre?
3. **¿Cómo le explicaríamos a alguien** la diferencia entre `WHERE` y `$match`?

Antes de cerrar:

- **Guardad el cuaderno** (en Colab, *Archivo → Guardar una copia en Drive*).
- Respondedlo con vuestras palabras en el **cuaderno de aprendizaje** (apartado 8 del cuaderno).
- **No subáis a ningún repositorio** una cadena de conexión real de MongoDB (lleva vuestra contraseña).

---

8. Hoja de registro de la pareja

Copiadla y rellenadla mientras trabajáis (una por pareja).

Pareja: ____ · Fecha: __

Parte	Ejercicios hechos	Quién conducía	Lo que más nos costó (y por qué)	Lo que hemos aprendido
Contar y filtrar (1 a 3)				
Actualizar y borrar (4 y 5)				
Agregaciones (6 y 7)				
Mini-reto SQL y MongoDB				

Los tres errores en los que más caímos: 1. 2. 3.

Dudas para preguntar al docente:

Para repasar la teoría

Si algo de la guía no os suena, repasad los apuntes del Sprint 3 (**Día 2 · NoSQL** y la sección «¿SQL o NoSQL?»). Si queréis probar **MongoDB de verdad** (Compass o Atlas), seguid la guía de MongoDB. Para comparar con SQL, tenéis la guía de SQL en parejas.